LeetCode Practice

12 Hard: DFS

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

12
HARD

Patterns (in order)

1. DFS (12)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

12 questions | tap a title to jump to it

Hard questions (12) Go >

DFS (12) Go >

H	1. Alien Dictionary	LC 269
H	2. Longest Increasing Path in a Matrix	LC 329
H	3. Redundant Connection II	LC 685
H	4. Cracking the Safe	LC 753
H	5. Making A Large Island	LC 827
H	6. Minimize Malware Spread	LC 924
H	7. Escape a Large Maze	LC 1036
H	8. Critical Connections in a Network	LC 1192
H	9. Sort Items by Groups Respecting Dependencies	LC 1203
H	10. Valid Arrangement of Pairs	LC 2097
H	11. Build a Matrix With Conditions	LC 2392
H	12. Greatest Common Divisor Traversal	LC 2709

Hard questions (12)

HARD | TOPIC 1 OF 1

DFS

12 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Alien Dictionary LC 269
2. Longest Increasing Path in a Matrix LC 329
3. Redundant Connection II LC 685
4. Cracking the Safe LC 753
5. Making A Large Island LC 827
6. Minimize Malware Spread LC 924
7. Escape a Large Maze LC 1036
8. Critical Connections in a Network LC 1192
9. Sort Items by Groups Respecting Dependencies LC 1203
10. Valid Arrangement of Pairs LC 2097
11. Build a Matrix With Conditions LC 2392
12. Greatest Common Divisor Traversal LC 2709

HARD | DFS | LeetCode #269 | 1 of 12 | 1/12

Alien Dictionary

QUESTION

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions, return any of them.

Example 1	
Input	`words = ["wrt","wrf","er","ett","rftt"]`
Output	`"wertf"`

Example 2	
Input	`words = ["z","x"]`
Output	`"zx"`

Example 3	
Input	`words = ["z","x","z"]`
Output	`""`
Explanation	*The order is invalid, so return "".*

CONSTRAINTS

- 1 <= words.length <= 100
- 1 <= words[i].length <= 100
- words[i] consists of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Given words sorted in alien lexicographic order, deduce the letter ordering.

Return any valid ordering, or '' if impossible (cycle or contradiction). Right?

Contradiction: longer word precedes its prefix (e.g. ['ab','a'])."

Are there duplicate elements? Should the sort be stable?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for ordering in orderings(letters):
    check that the words are sorted under it
return the first that works
```

TIME	SPACE
O(L! * n * m)	O(L)

WHY IT IS SLOW

L! orderings (L distinct letters) tested against all words. Build 'a before b' edges from adjacent words and topologically sort: O(total characters).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (Kahn's BFS topological sort)

"Build order graph from word comparisons, then Kahn's BFS topo sort — O(n_chars + E)."

Step 1 — Extract ordering constraints:

Compare adjacent words char by char. First differing position gives edge: earlier_char -> later_char.

Special case: if word[i] is a prefix of word[i+1] but longer -> invalid -> return ''.

Step 2 — Kahn's BFS topological sort on the 26-char alphabet:

Build adjacency list and indegree map for each unique character seen.

Enqueue all chars with indegree 0.

BFS: pop char, append to result, decrement neighbors' indegrees, enqueue if 0.

Step 3 — Validate:

If result length < unique chars -> cycle -> return ''.

HOW we handle characters not constrained: all unique chars not in any comparison pair

go in at indegree 0 and can appear anywhere in output (any valid topo order accepted).

Time: O(total_chars + unique_chars). Space: O(26²)."

Key insight: adjacent word comparisons yield at most one ordering constraint per pair — building the graph from these edges and running topological BFS gives O(total_chars).

PHASE 4 CLEAN CODE

```
class _269_AlienDictionary:
    def alienOrder(self, words: List[str]) -> str:
        graph   = defaultdict(set)
        indegree = {c: 0 for w in words for c in w}

        for i in range(len(words) - 1):
            w1, w2 = words[i], words[i + 1]
            min_len = min(len(w1), len(w2))
            if len(w1) > len(w2) and w1[:min_len] == w2[:m
in_len]:
                return ''                          # inval
id: prefix ordering
            for c1, c2 in zip(w1, w2):
                if c1 != c2:
                    if c2 not in graph[c1]:
```

```
                    graph[c1].add(c2)
                    indegree[c2] += 1
                break

        queue  = deque(c for c in indegree if indegree[c]
== 0)
        result = []
        while queue:
            c = queue.popleft()
            result.append(c)
            for nb in graph[c]:
                indegree[nb] -= 1
                if indegree[nb] == 0:
                    queue.append(nb)

        return ''.join(result) if len(result) == len(indeg
ree) else ''
```

PHASE 5 TEST & DEBUG

"Let me trace alienOrder on words=['wrt','wrf','er','ett','rftt']."

Extract ordering from adjacent words:

'wrt'vs'wrf': 't'<'f'. 'wrf'vs'er': 'w'<'e'. 'er'vs'ett': 'r'<'t'. 'ett'vs'rftt': 'e'<'r'.

Edges: t->f, w->e, r->t, e->r.

Topological sort: in_degree: f=1,e=1,t=1,r=1. w=0 (no incoming).

BFS: queue=[w]. Pop w->order=[w]. decrement e->in_degree[e]=0->enqueue.

Pop e->order=[we]. decrement r->0->enqueue. Pop r->order=[wer]. decrement t->0->enqueue.

Pop t->order=[wert]. decrement f->0->enqueue. Pop f->order=[wertf] ok

Return 'wertf'. WHY topo sort: only valid orderings are those consistent with all constraints.

DFS topo sort on char graph: visited['a']=in-stack. call stack: dfs('a')->dfs('b')->dfs('c')->push c done. pop stack gives reverse topo order. visited prevents revisit.

DAG (Directed Acyclic Graph): edges = char ordering constraints. component: cycle in graph -> invalid (alien dictionary). find(node)->topo index. dist[char]=position in final alphabet.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(C)	**O(1)**
where C = total characters	at most 26 letters

The prefix check catches the only invalid case that doesn't manifest as a graph cycle.

Given more time I'd ask: return ALL valid orderings?

Backtracking on the topological sort — collect all linearisations.

HARD | DFS | LeetCode #329 | 2 of 12 | 2/12

Longest Increasing Path in a Matrix

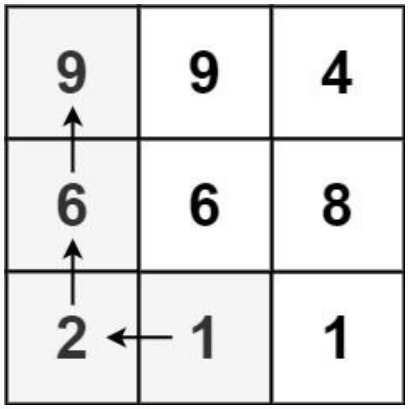
QUESTION

Given an m x n integers matrix, return the length of the longest increasing path in matrix.

From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

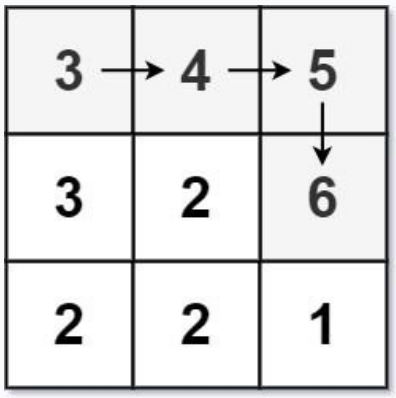
Example 1

Input	`matrix = [[9,9,4],[6,6,8],[2,1,1]]`
Output	`4`
Explanation	*The longest increasing path is [1, 2, 6, 9].*



Example 2

Input	`matrix = [[3,4,5],[3,2,6],[2,2,1]]`
Output	`4`
Explanation	*The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.*



Example 3

Input	`matrix = [[1]]`
Output	`1`

CONSTRAINTS

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 200
- 0 <= matrix[i][j] <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Longest strictly increasing path (4-directional). Return path length. Right?

Values can repeat but path must be strictly increasing."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    DFS along strictly increasing neighbours,
        recomputing every path
    best = max(best, candidate)
```

TIME	SPACE
exponential in m * n	O(m * n)

WHY IT IS SLOW

The same cell is re-explored from every path that reaches it. Memoising the longest path starting at each cell (or processing cells by increasing value) is O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (DFS + memoization)

"DFS with memoization on each cell — already O(mn). Walk-through:"

`dfs(r, c):`

If memo[r][c] != 0: return memo[r][c] <- cached result, skip recomputation.

`best = 1 <- path starting at (r,c) is at least length 1 (just this cell).`

For each of 4 directions (dr, dc):

(nr, nc) = (r+dr, c+dc)

If in bounds AND grid[nr][nc] > grid[r][c]: <- strictly increasing

`best = max(best, dfs(nr, nc) + 1)`

<- if path starting at neighbor has length L, path through (r,c) is L+1.

`memo[r][c] = best; return best.`

WHY no visited set: strictly increasing values prevent cycles — you can never return

to a cell you came from (it would have a smaller value, violating the condition).

Memoization: each cell computed once -> O(mn) total.

PHASE 3 OPTIMIZE (DFS + memoization)

Time: O(mn). Space: O(mn)."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _329_LongestIncPathMatrix:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        m, n  = len(matrix), len(matrix[0])
        memo  = {}

        def dfs(r, c):
            if (r, c) in memo: return memo[(r, c)]
            best = 1
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and matrix[nr][nc] > matrix[r][c]:
                    best = max(best, dfs(nr, nc) + 1)
            memo[(r, c)] = best
            return best

        return max(dfs(r, c) for r in range(m) for c in range(n))
```

PHASE 5 TEST & DEBUG

"Let me trace longestIncreasingPath on [[9,9,4],[6,6,8],[2,1,1]]."

DFS+memo from each cell, move to strictly larger neighbors

From (2,0)=2: can go to (1,0)=6->(0,0)=9 (len=3) or (0,0) directly (len=2)

memo[2][0]=3, memo[1][0]=2, memo[0][0]=1

From (2,1)=1: neighbors (2,0)=2>1 -> memo[2][0]=3 -> memo[2][1]=4

`max_path = 4 ok`

WHY memoization: each cell computed once; DFS explores monotone paths (no cycles possible since strictly increasing).

Topological order: cells with no larger neighbor = length 1.

DFS+memo: visited implicitly (strictly increasing path cannot revisit). call stack: dfs(i,j)->dfs(neighbors with larger value). dp[i][j]=1+max(dfs(valid neighbor)).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** for memo. No visited set needed — strict increase prevents cycles

Topological sort alternative: build DAG (Directed Acyclic Graph) (only edges to larger neighbours), count levels.

Same O(m*n) but iterative.

Given more time I'd ask: longest path with values differing by at most 1?

Need a visited set now (can revisit same value) — different problem.

IN YOUR APP | Expose | Editorial | Community

HARD | DFS | LeetCode #685 | 3 of 12 | 3/12

Redundant Connection II

QUESTION

In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

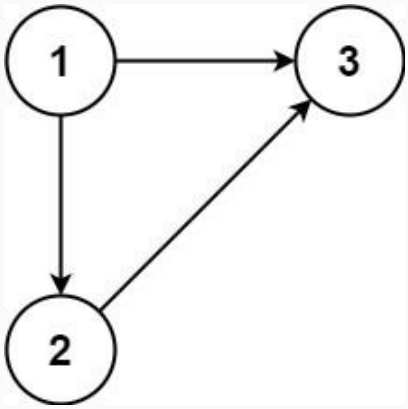
The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n, and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u_i, v_i]$ that represents a directed edge connecting nodes u_i and v_i, where u_i is a parent of child v_i.

Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

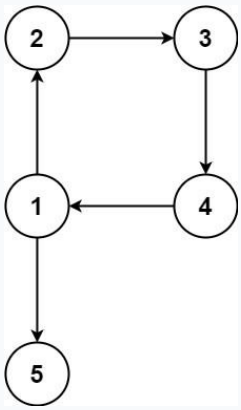
Example 1

Input	`edges = [[1,2],[1,3],[2,3]]`
Output	`[2,3]`



Example 2

Input	`edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]`
Output	`[4,1]`



CONSTRAINTS

QUESTION

- n == edges.length
- 3 <= n <= 1000
- edges[i].length == 2
- $1 <= u_i, v_i <= n$
- $u_i != v_i$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Directed graph: rooted tree + one extra edge. Find the edge to remove.

If multiple answers, return the last one in input. Right?

Extra edge causes: (1) a node with two parents, OR (2) a cycle, OR both."

Should I handle disconnected graphs or graphs with no edges?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for edge in range(the_last,_the_first_+_1):
    remove it and check whether what remains is a
        rooted tree
return the first edge whose removal works
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A full validity check per edge. Find a node with two parents (two candidate edges) and use union-find to decide which one to drop (or detect the cycle edge): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (three-case analysis + Union-Find)

Approach: optimal approach

"Track in-degree: if any node has two parents, record both candidate edges.

Then run Union-Find ignoring the second parent candidate.

Case 1 (two parents, no cycle in remaining): remove second candidate.

Case 2 (two parents, cycle in remaining): remove first candidate.

Case 3 (no node with two parents): cycle exists -> last edge forming cycle."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

Key insight: union-find with in-degree tracking covers three cases: only double-parent, only cycle, or both — the first edge violating either condition is the answer.

PHASE 4 CLEAN CODE

Find if any node has two parents

```
class _685_RedundantConnectionII:
    def findRedundantDirectedConnection(self, edges: List[
List[int]]) -> List[int]:
        n      = len(edges)
        parent = list(range(n + 1))
        cand1  = cand2 = None

        parent_map = {}
        for u, v in edges:
            if v in parent_map:
                cand1 = parent_map[v]   # first edge givin
g v a parent
                cand2 = [u, v]          # second edge givi
ng v another parent
                break
            parent_map[v] = [u, v]

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(u, v) -> bool:
            pu, pv = find(u), find(v)
            if pu == pv: return False
            parent[pu] = pv
            return True

        parent = list(range(n + 1))
        for u, v in edges:
            if [u, v] == cand2:        # skip the second c
andidate
                continue
            if not union(u, v):
                return cand1 if cand1 else [u, v]

        return cand2
```

PHASE 5 TEST & DEBUG

"Let me trace findRedundantDirectedConnection on edges=[[1,2],[1,3],[2,3],[3,1]]."

Step 1: find nodes with in-degree 2 (two parents). [1,2]: 1->2,1->3: node 3 gets parent 1. [2,3]: 2->3. Node 3 has two parents (1 and 2) -> candidates = [1,3] and [2,3].

Step 2: if two-parent node found, try removing each candidate:

Remove [2,3]: check if remaining edges form a valid tree (connected, no cycle).

[1,2],[1,3],[3,1]: 3->1->3: cycle! Not valid.

Remove [1,3]: check [1,2],[2,3],[3,1]: 1->2->3->1: cycle! Not valid either.

Step 3: if no two-parent node: pure cycle -> return last edge in cycle.

Here: [3,1] creates cycle 1->3->1 -> return [3,1] ok

WHY two cases: directed tree violations are either 'two parents' or 'pure cycle'.

Union-Find/DFS: find(child) checks if adding parent creates cycle. visited used in DFS to find which candidate edge to remove. component merging identifies redundant parent.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * alpha(n))	**O(n)** Three cases arise from two root causes: double parent and cycle

The Union-Find handles both: cycle detection when cand2 is skipped tells us which candidate to remove.

Given more time I'd ask: undirected version?

That's Redundant Connection (#684) — standard Union-Find, return first edge that creates a cycle.

IN YOUR APP | Expose | Editorial | Community

HARD | DFS | LeetCode #753 | 4 of 12 | 4/12

Cracking the Safe

QUESTION

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range [0, k - 1].

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the most recent n digits that were entered each time you type a digit.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the shortest string that contains all k^n combinations of n digits from [0..k-1] as substrings (de Bruijn sequence). Right?"

"A few quick questions:

The result is cyclic — we work with a graph where edges are n-grams and find an Eulerian circuit."

"Let me walk: n=1, k=2 -> '01' (all single digits 0,1) ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for string in strings:
    # of length k^n + n - 1 over the k digits
    if all its length-n windows are distinct:
        return it
```

TIME	SPACE
O(k^(n * k^n))	O(n * k^n)

WHY IT IS SLOW

Tries astronomically many strings. The password is a de Bruijn sequence: an Eulerian cycle through all (n - 1)-digit states (Hierholzer's DFS, or the prefer-largest greedy) builds it in O(k^n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Hierholzer's Eulerian circuit on de Bruijn graph — O(k^n). Walk-through:"

de Bruijn graph: node = (n-1)-digit string. Edge a->b if a[1:]+digit = b (append one digit).

An Eulerian circuit visits every EDGE exactly once = every n-length combination visited.

Build complete graph (every (n-1)-gram node connects to k nodes via k digits).

PHASE 3 OPTIMIZE

Find Eulerian circuit using Hierholzer's:

DFS from '0'*(n-1). Post-order append to result. Reverse.

Extract the safe code: each edge's new digit appended -> sequence of k^n digits.

Return first n-1 chars + all edge digits (or just Euler circuit result).

WHY de Bruijn: every n-length string is exactly one edge -> Eulerian circuit = exhaustive.

Time: O(k^n). Space: O(k^n)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _753_CrackingTheSafe:
    def crackSafe(self, n, k):
        start = '0' * n
        visited = set([start])
        result = [start]
        total = k ** n
        def dfs(node):
            if len(visited) == total:
                return True
            for c in map(str, range(k)):
                next_node = node[1:] + c
                if next_node not in visited:
                    visited.add(next_node)
                    result.append(c)
                    if dfs(next_node):
                        return True
                    result.pop()
                    visited.remove(next_node)
            return False
        dfs(start)
        return ''.join(result)
```

PHASE 5 TEST & DEBUG

"Let me trace crackSafe(n=1, k=2) expected '10' or '01' (de Bruijn sequence)."

Eulerian circuit on de Bruijn graph:

Nodes: strings of len n-1=0 -> single node '' (for n=1)

Edges: '0', '1' (each k-mer)

Start with '0'*(n-1)='' -> add '0': seen={'0'}. From '' try '0': already seen. Try '1': add -> '01'.

'01' contains both k-mers '0' and '1' -> length=n+k^n-1=1+2-1=2 ok

For n=2,k=2: de Bruijn seq '00110' covers '00','01','11','10' -> all 2-mers ok

WHY Eulerian circuit: each edge=one new symbol; visiting all edges = all combinations covered.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k^n * n)	O(k^n)

Follow-up: Valid Arrangement of Pairs (#2097) — same Eulerian path/circuit idea.

Follow-up: if n=1, trivially concatenate all digits.

IN YOUR APP | Expose | Editorial | Community

HARD | DFS | LeetCode #827 | 5 of 12 | 5/12

Making A Large Island

QUESTION

You are given an n x n binary matrix grid. You are allowed to change at most one 0 to be 1.

Return the size of the largest island in grid after applying this operation.

An island is a 4-directionally connected group of 1s.

Example 1

Input	`grid = [[1,0],[0,1]]`
Output	`3`
Explanation	*Change one 0 to 1 and connect two 1s, then we get an island with area = 3.*

Example 2

Input	`grid = [[1,1],[1,0]]`
Output	`4`
Explanation	*Change the 0 to 1 and make the island bigger, only one island with area = 4.*

Example 3

Input	`grid = [[1,1],[1,1]]`
Output	`4`
Explanation	*Can't change any 0 to 1, only one island with area = 4.*

CONSTRAINTS

- n == grid.length
- n == grid[i].length
- 1 <= n <= 500
- grid[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can flip exactly one 0 to 1. Return the size of the largest island after the flip. Right?"

"A few quick questions:

Each island's size must be computed; then for each 0, sum adjacent island sizes + 1.

Islands might touch the same 0 — avoid double-counting by using island IDs."

"Let me walk: grid=[[1,0],[0,1]] -> flip any 0 -> 3 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    flip it to 1, flood-fill the island
        containing it and measure, then flip back
    best = max(best, candidate)
```

TIME	SPACE
O(n^4)	O(n^2)

WHY IT IS SLOW

A fresh flood fill for every zero. Label every island once and store its size; for each 0 add the sizes of the distinct neighbouring islands plus one: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Union-Find + try adding each water cell — O(mn * alpha(mn)). Walk-through:"*

Phase 1: Union-Find all land cells. Track size of each component.

Phase 2: For each water cell (0), try flipping to land:

Gather all DISTINCT neighbor components (by their roots).

`merged_size = 1 (the new cell) + sum of distinct neighbor component sizes.`

`best = max(best, merged_size).`

Return best.

WHY distinct roots: multiple neighbors may belong to same island -> avoid double-counting.

WHY enumerate water cells (not land): we're trying one flip -> O(mn) iterations.

WHY Union-Find over BFS: Union-Find gives O(alpha) per merge; BFS would be O(mn) per flip.

Time: O(mn * alpha(mn)). Space: O(mn)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _827_MakingALargeIsland:
    def largestIsland(self, grid):
        n = len(grid)
        island_id = 2
        island_size = {}
        def dfs(r, c, iid):
            if not (0 <= r < n and 0 <= c < n) or grid[r][
c] != 1:
                return 0
            grid[r][c] = iid
            size = 1
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                size += dfs(r+dr, c+dc, iid)
            return size
        for r in range(n):
```

```
            for c in range(n):
                if grid[r][c] == 1:
                    island_size[island_id] = dfs(r, c, isl
and_id)
                    island_id += 1
        best = max(island_size.values(), default=0)
        for r in range(n):
            for c in range(n):
                if grid[r][c] == 0:
                    seen = set()
                    total = 1
                    for dr, dc in [(0,1),(0,-1),(1,0),(-1,
0)]:
                        nr, nc = r+dr, c+dc
                        if 0 <= nr < n and 0 <= nc < n and
 grid[nr][nc] > 1:
                            seen.add(grid[nr][nc])
                    for iid in seen:
                        total += island_size[iid]
                    best = max(best, total)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace largestIsland on [[1,0],[0,1]]."

Step 1: label each island with unique ID and size. Island starting at (0,0): label=2, size=1. (1,1): label=3, size=1.

Step 2: try flipping each 0:

(0,1): neighbors (0,0)=label2, (1,1)=label3. Distinct labels: {2,3}. size=1+1+1=3.

(1,0): neighbors (0,0)=label2, (1,1)=label3. size=1+1+1=3.

Max: 3 ok

[[1,1],[1,1]]: all 1s -> no 0 to flip -> max island = 4 ok (size of entire grid).

WHY label islands first: O(1) lookup of each neighbor's island size when checking flips.

WHY distinct labels: avoid double-counting islands that are already connected.

DFS: visited={(r,c)}. Label each island component with ID. Store size. Flip each 0-cell: add 1 + sum of adjacent distinct component sizes. visited set built per DFS call.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
two passes over the grid	for island labels

Follow-up: flipping k zeros — extension to k is NP (Non-deterministic Polynomial)-hard without constraints.

Follow-up: Max Area of Island (#695) — same DFS, no flipping needed.

HARD | DFS | LeetCode #924 | 6 of 12 | 6/12

Minimize Malware Spread

QUESTION

You are given a network of n nodes represented as an n x n adjacency matrix graph, where the ith node is directly connected to the jth node if graph[i][j] == 1.

Some nodes initial are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose M(initial) is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will remove exactly one node from initial.

Return the node that, if removed, would minimize M(initial). If multiple nodes could be removed to minimize M(initial), return such a node with the smallest index.

Note that if a node was removed from the initial list of infected nodes, it might still be infected later due to the malware spread.

Example 1

Input	`graph = [[1,1,0],[1,1,0],[0,0,1]], initial = [0,1]`
Output	`0`

Example 2

Input	`graph = [[1,0,0],[0,1,0],[0,0,1]], initial = [0,2]`
Output	`0`

Example 3

Input	`graph = [[1,1,1],[1,1,1],[1,1,1]], initial = [1,2]`
Output	`1`

CONSTRAINTS

- n == graph.length
- n == graph[i].length
- 2 <= n <= 300
- graph[i][j] is 0 or 1.
- graph[i][j] == graph[j][i]
- graph[i][i] == 1
- 1 <= initial.length <= n
- 0 <= initial[i] <= n - 1
- All the integers in initial are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove one node to minimise the spread of malware. Malware in initial list spreads to connected nodes.

Return the node whose removal minimises final infection count. Ties: smallest index. Right?"

"A few quick questions:

Only one removal? Yes. Output the removed node index."

"Let me walk: graph=[[1,1,0],[1,1,0],[0,0,1]], initial=[0,1] -> remove 0 -> 1 infected? Remove either 0 or 1 leaves the other still connected -> still 2. Return 0 (smaller) -> 0 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    remove it, simulate the malware spreading
        (BFS) and count the infected nodes
    best = max(best, removal)
```

TIME	SPACE
O(k * n^2)	O(n^2)

WHY IT IS SLOW

A full spread simulation per removal. Find the connected components once (union-find): removing a node only helps if it is the only initial node in its component, saving that component's size.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Union-Find: find initial infected node's component, count saves — O(n² * alpha(n)). Walk-through:"*

Union-Find to identify connected components.

For each node with initial malware: identify its component.

If a component has exactly 1 initially-infected node: removing that node saves

the entire component from infection. Count = component size.

If 2+ initially-infected nodes in same component: removing one still leaves the other

to infect the component -> saves 0.

Answer = node in a single-infected component with the LARGEST component size.

If tie: return smallest node index.

If no such component: return smallest initial node (removing any saves 0).

Time: O(n² * alpha(n)). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _924_MinimizeMalwareSpread:
    def minMalwareSpread(self, graph, initial):
        n = len(graph)
        parent = list(range(n))
        def find(x):
            while parent[x] != x: parent[x] = parent[paren
t[x]]; x = parent[x]
            return x
        def union(x, y):
            parent[find(x)] = find(y)
        for i in range(n):
            for j in range(i+1, n):
                if graph[i][j]:
                    union(i, j)
        component_size = {}
        for i in range(n):
            r = find(i)
            component_size[r] = component_size.get(r, 0) +
 1
        initial_per_component = {}
        for node in initial:
            r = find(node)
            initial_per_component[r] = initial_per_compone
nt.get(r, 0) + 1
        result = min(sorted(initial), key=lambda node: (
            -component_size[find(node)] if initial_per_com
ponent.get(find(node), 0) == 1 else 0,
            node
        ))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace minMalwareSpread on graph=[[1,1,0],[1,1,0],[0,0,1]], initial=[0,1]."

Union-Find: component(0)=component(1) (edge between them). component(2)=isolated.

For each initial node: which component does it belong to?

Node 0: component A (size=2). Node 1: same component A.

Component A has 2 initial nodes -> removing either doesn't help (the other still infects A).

No component has exactly 1 initial node -> removing any saves 0 nodes.

Return smallest initial node = 0 ok (per problem: return min index when tie in savings).

initial=[0,2]: component A has node 0 (1 initial), component isolated has node 2 (1 initial).

Remove 0: saves size(A)=2. Remove 2: saves 1. Return 0 ok

Union-Find: find(node)->root of component. union(u,v) merges connected nodes. component_size[root] tracked. Initial nodes in same component share infection. visited: all nodes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$ adjacency matrix union-find	$O(n)$

Follow-up: Minimize Malware Spread II (#928) — remove and reconnect; different approach.

Follow-up: Friend Circles (#547) — same union-find component counting.

IN YOUR APP | Expose | Editorial | Community

HARD | DFS | LeetCode #1036 | 7 of 12 | 7/12

Escape a Large Maze

QUESTION

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y).

We start at the source = [s_x, s_y] square and want to reach the target = [t_x, t_y] square. There is also an array of blocked squares, where each blocked[i] = [x_i, y_i] represents a blocked square with coordinates (x_i, y_i).

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1

Input	`blocked = [[0,1],[1,0]], source = [0,0], target = [0,2]`
Output	`false`
Explanation	*The target square is inaccessible starting from the source square because we cannot move.* *We cannot move north or east because those squares are blocked.* *We cannot move south or west because we cannot go outside of the grid.*

Example 2

Input	`blocked = [], source = [0,0], target = [999999,999999]`
Output	`true`
Explanation	*Because there are no blocked cells, it is possible to reach the target square.*

CONSTRAINTS

- 0 <= blocked.length <= 200
- blocked[i].length == 2
- 0 <= x_i, y_i < 10^6
- source.length == target.length == 2
- 0 <= s_x, s_y, t_x, t_y < 10^6
- source != target
- It is guaranteed that source and target are not blocked.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

1M×1M grid. Up to 200 blocked cells. Can source reach target? Right?

With at most 200 blocked cells in a semicircle, max enclosed area ~ 200²/2 = 20,000 cells.

So if BFS from source visits > 20,000 cells without finding target, source is NOT enclosed.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
BFS from the source over the whole 10^6 x 10^6 grid,
    avoiding blocked cells, until the target is
    reached
```

TIME	SPACE
O(10^12)	O(10^12)

WHY IT IS SLOW

Explores up to a trillion cells. With at most B blockers they can enclose at most B(B - 1) / 2 cells, so a BFS from each end that stops after exceeding that many cells (or reaching the other end) is O(B^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (BFS with cell limit)

"Limited BFS from BOTH endpoints — O(n²) where n=len(blocked). Walk-through:"

Key insight: blocked cells can form an enclosing shape. Max area they can enclose

= n*(n-1)//2 (triangular number). Call this max_cells.

Run BFS from source: if BFS visits > max_cells cells OR reaches target -> source not enclosed.

Run BFS from target: if BFS visits > max_cells cells OR reaches source -> target not enclosed.

If BOTH BFS runs escape (visit > max_cells) -> no enclosure -> return True.

If either BFS terminates early (trapped in < max_cells cells) -> enclosed -> return False.

WHY check both endpoints: source could be free but target enclosed, or vice versa.

WHY max_cells bound: n blocked cells can enclose at most n*(n-1)//2 area — any BFS

exceeding this count proves the start isn't trapped.

Time: O(n²) where n=len(blocked). Space: O(n²).

PHASE 4 CLEAN CODE

```python
class _1036_EscapeLargeMaze:
    def isEscapePossible(self, blocked: List[List[int]], s
ource: List[int], target: List[int]) -> bool:
        if not blocked:
            return True
        n          = len(blocked)
        max_cells = n * (n + 1) // 2
        blocked_set = {(r, c) for r, c in blocked}

        def bfs(start, end):
            queue   = deque([tuple(start)])
            visited = {tuple(start)}
            while queue:
                r, c = queue.popleft()
                if [r, c] == end:
```

```
                    return True
                if len(visited) > max_cells:
                    return True   # too many cells explore
d -> not enclosed
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < 10**6 and 0 <= nc < 10**6
 and (nr,nc) not in blocked_set and (nr,nc) not in visited
:
                        visited.add((nr, nc))
                        queue.append((nr, nc))
            return False

        return bfs(source, target) and bfs(target, source)
```

PHASE 5 TEST & DEBUG

"Let me trace isEscapePossible(maze=[], source=[0,0], target=[999999,999999])."

Key insight: blocked cells at most 200. Max enclosable area = 200*200/2 = 20000

BFS from source: if we reach target OR explore >20000 cells -> can escape from source

BFS from target: same check from the other side

If both BFS succeed (explore >bound or reach other side's start) -> True ok

If either gets trapped in < bound cells -> False

WHY bound check: 200 blocked cells can enclose at most ~20000 area. Exceeding = escaped the potential cage.

Edge: source==target -> True; no blocked cells -> always True.

BFS: queue=[(0,0,0)] (r,c,steps). visited={(0,0)}. Expand up to bound=200*200/2=20000 cells. If |visited|>bound OR target reached -> can escape. Run BFS from source and target.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
per BFS (at most n*(n+1)/2 cells)	n <= 200 -> fast

The dual-BFS is essential: source might not be enclosed but target might be.

Given more time I'd ask: what if blocked cells can be added dynamically?

Re-run BFS on each addition — or maintain Union-Find of open cells.

HARD | DFS | LeetCode #1192 | 8 of 12 | 8/12

Critical Connections in a Network

QUESTION

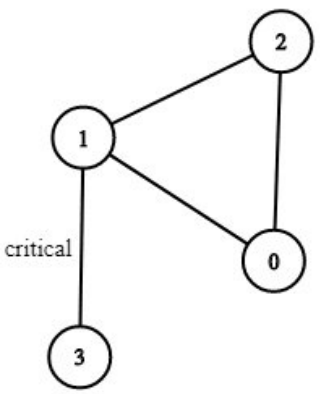
There are n servers numbered from 0 to n - 1 connected by undirected server-to-server connections forming a network where connections[i] = [a_i, b_i] represents a connection between servers a_i and b_i. Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1

Input	`n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]`
Output	`[[1,3]]`
Explanation	*[[3,1]] is also accepted.*



Example 2

Input	`n = 2, connections = [[0,1]]`
Output	`[[0,1]]`

CONSTRAINTS

- 2 <= n <= 10^5
- n - 1 <= connections.length <= 10^5
- 0 <= a_i, b_i <= n - 1
- a_i != b_i
- There are no repeated connections.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Critical connection (bridge) = edge whose removal disconnects the graph. Right?

Use Tarjan's bridge-finding algorithm."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for connection in connections:
    remove it and check whether the network is
        still connected (a fresh DFS)
    collect those that disconnect it
```

TIME	SPACE
O(E * (V + E))	O(V + E)

WHY IT IS SLOW

A graph search per edge. One DFS keeping discovery times and low-link values (Tarjan): an edge (u, v) is a bridge when low[v] > disc[u]: O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (Tarjan's DFS)

"Tarjan's bridge-finding DFS — already O(V+E). Walk-through of how disc and low work:"

`disc[u] = discovery time when u is first visited (increases with each new node visited).`

`low[u] = earliest disc time reachable from u's subtree via back-edges (NOT through parent edge).`

`dfs(u, parent):`

`disc[u] = low[u] = timer++`

`for each neighbor v:`

`if v == parent: skip <- avoid going back the way we came (undirected graph)`

`if not visited:`

`dfs(v, u)`

`low[u] = min(low[u], low[v]) <- v might reach back to an ancestor of u`

`if low[v] > disc[u]: <- v can't escape u's subtree without using edge (u,v)`

`bridges.append([u,v]) <- (u,v) is a bridge!`

`else:`

`low[u] = min(low[u], disc[v]) <- back-edge: v is already visited ancestor`

WHY low[v] > disc[u] means bridge: if v could reach u (or u's ancestor) via another path,

low[v] <= disc[u]. Only if low[v] > disc[u] is v 'trapped' -> removing (u,v) disconnects v.

Time: O(V+E). Space: O(V+E)."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```python
class _1192_CriticalConnections:
    def criticalConnections(self, n: int, connections: Lis
t[List[int]]) -> List[List[int]]:
        graph = defaultdict(list)
        for u, v in connections:
            graph[u].append(v)
            graph[v].append(u)
        disc = [-1] * n
        low  = [-1] * n
        timer = [0]
        result = []

        def dfs(node, parent):
            disc[node] = low[node] = timer[0]
            timer[0] += 1
            for nb in graph[node]:
                if nb == parent: continue
                if disc[nb] == -1:
                    dfs(nb, node)
                    low[node] = min(low[node], low[nb])
                    if low[nb] > disc[node]:
                        result.append([node, nb])
                else:
                    low[node] = min(low[node], disc[nb])

        dfs(0, -1)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace criticalConnections (Tarjan bridges) on n=4, connections=[[0,1],[1,2],[2,0],[1,3]]."

DFS from 0 (parent=-1). disc/low initialized. timer=0.

Visit 0: disc[0]=low[0]=0, timer=1 -> visit neighbors 1,2

Visit 1 (parent=0): disc[1]=low[1]=1, timer=2 -> visit 2,3

Visit 2 (parent=1): disc[2]=low[2]=2, timer=3 -> back-edge to 0: low[2]=min(2,disc[0])=0

Return to 1: low[1]=min(1,low[2])=0 -> Visit 3 (parent=1): disc[3]=low[3]=3. No neighbors.

Back at 1: low[3]=3 > disc[1]=1 -> BRIDGE [1,3] found ok

Triangle 0-1-2: low[2]=0 <= disc[1]=1 -> NOT a bridge ok

Return [[1,3]] ok

WHY low values: low[v]=earliest disc reachable via back-edges below v. low[v]>disc[u] means no shortcut.

Bridge detection: component split. Removing bridge creates 2+ components. find(root)->DFS discovery. component 1={0,1,2}, component 2={3}. Bridge [1,3]: dist=1 apart in DFS tree ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	**O(V+E)** Tarjan's bridge algorithm in one DFS pass

low[v] > disc[u] means v's subtree can't 'back-connect' to u's subtree — edge is a bridge.

Given more time I'd ask: articulation points instead of bridges?

PHASE 6 COMPLEXITY & FOLLOW-UP

Similar but: u is an articulation point if any child v has low[v] >= disc[u] (and u is not root).

IN YOUR APP | Expose | Editorial | Community

HARD | DFS | LeetCode #1203 | 9 of 12 | 9/12

Sort Items by Groups Respecting Dependencies

QUESTION

There are n items each belonging to zero or one of m groups where group[i] is the group that the i-th item belongs to and it's equal to -1 if the i-th item belongs to no group. The items and the groups are zero indexed. A group can have no item belonging to it.

Return a sorted list of the items such that:

CONSTRAINTS

- The items that belong to the same group are next to each other in the sorted list.
- There are some relations between these items where beforeItems[i] is a list containing all the items that should come before the i-th item in the sorted array (to the left of the i-th item).

Return any solution if there is more than one solution and return an empty list if there is no solution.

Example 1

Input	`n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3,6],[],[],[]]`
Output	`[6,3,4,1,5,2,0,7]`

Item	Group	Before
0	-1	
1	-1	6
2	1	5
3	0	6
4	0	3, 6
5	1	
6	0	
7	-1	

Example 2

Input	`n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3],[],[4],[]]`
Output	`[]`
Explanation	*This is the same as example 1 except that 4 needs to be before 6 in the sorted list.*

- $1 <= m <= n <= 3 * 10^4$
- group.length == beforeItems.length == n
- 1 <= group[i] <= m - 1
- 0 <= beforeItems[i].length <= n - 1
- 0 <= beforeItems[i][j] <= n - 1
- i != beforeItems[i][j]

QUESTION

- beforeItems[i] does not contain duplicates elements.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Items belong to groups. Within a group, order matters (beforeItems dependencies).

Also groups must respect inter-group dependencies. Return a valid topological order or []. Right?"

"A few quick questions:

Items without a group get their own group? Yes — assign unique group IDs."

"Let me walk: n=6, m=2, group=[3,-1,0,0,1,0], beforeItems=[[],[],[1],[6],[3,6],[2]] -> valid topo ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(items):
    if every group is contiguous and every
        dependency is respected:
        return it
```

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

n! orderings. Topologically sort the groups (from the dependencies between items in different groups), then topologically sort the items inside each group: O(n + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-level topological sort — already O(n+E). Walk-through:"

Level 1: topological sort of ITEMS within each group.

For each group, collect item dependencies. Kahn's BFS within the group -> item order.

Level 2: topological sort of GROUPS.

Build group dependency graph: if item A -> item B and they're in different groups,

add group(A) -> group(B) edge.

Kahn's BFS on groups -> group order.

COMBINE: for each group in group-order, output its items in item-order.

WHY two passes: intra-group constraints (item ordering) and inter-group constraints

(group ordering) are independent and can be solved separately then merged.

PHASE 3 OPTIMIZE

If either topological sort is impossible (cycle) -> return [].

Time: O(n + E). Space: O(n + E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _1203_SortItemsByGroupsRespectingDependencies:
    def sortItems(self, n, m, group, beforeItems):
        for i in range(n):
            if group[i] == -1:
                group[i] = m
                m += 1
        item_adj = defaultdict(list)
        item_indeg = [0] * n
        grp_adj = defaultdict(list)
        grp_indeg = defaultdict(int)
        for g in set(group):
            grp_indeg[g] = grp_indeg.get(g, 0)
        for v in range(n):
            for u in beforeItems[v]:
                item_adj[u].append(v)
                item_indeg[v] += 1
                if group[u] != group[v]:
                    grp_adj[group[u]].append(group[v])
                    grp_indeg[group[v]] += 1
        def kahn(nodes, adj, indeg):
            queue = deque(x for x in nodes if indeg[x] ==
0)
            order = []
            while queue:
                x = queue.popleft()
                order.append(x)
                for y in adj[x]:
                    indeg[y] -= 1
                    if indeg[y] == 0:
                        queue.append(y)
            return order if len(order) == len(nodes) else
[]
        grp_order = kahn(list(set(group)), grp_adj, grp_in
deg)
        if not grp_order:
            return []
        item_order = kahn(list(range(n)), item_adj, item_i
ndeg)
        if not item_order:
            return []
        grp_items = defaultdict(list)
        item_pos = {x: i for i, x in enumerate(item_order)
}
        for i in range(n):
            grp_items[group[i]].append(i)
        result = []
        for g in grp_order:
            result.extend(sorted(grp_items[g], key=lambda
x: item_pos[x]))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace sortItems(n=6, m=2, group=[...], beforeItems=[...])."

Two-level topological sort: first within groups, then between groups

Assign each item without group its own unique group ID

PHASE 5 TEST & DEBUG

Build: (1) item->item edges within/between groups; (2) group->group edges

Topo sort items within each group; topo sort groups

If cycle in either item or group DAG (Directed Acyclic Graph) -> return [] (impossible)

Merge: interleave group outputs in topological group order ok

WHY two-level topo: inter-group ordering must respect group dependencies; intra-group respects item order.

Edge: empty beforeItems -> any valid group ordering works.

Two-level DAG: dist[item]=topo order within group. dist[group]=topo order of groups. cost=0 (BFS by dependency levels). component: each group = sub-graph. find(item)->group.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m+E) where E=total beforeItems edges	**O(n+m+E)**

Follow-up: Course Schedule II (#210) — single-level topological sort.

Follow-up: if items have weights, topological sort + DP for shortest path.

HARD | DFS | LeetCode #2097 | 10 of 12 | 10/12

Valid Arrangement of Pairs

QUESTION

You are given a 0-indexed 2D integer array pairs where pairs[i] = [$start_i$, end_i]. An arrangement of pairs is valid if for every index i where 1 <= i < pairs.length, we have end_{i-1} == $start_i$.

Return any valid arrangement of pairs.

Note: The inputs will be generated such that there exists a valid arrangement of pairs.

Example 1

Input	`pairs = [[5,1],[4,5],[11,9],[9,4]]`
Output	`[[11,9],[9,4],[4,5],[5,1]]`
Explanation	*This is a valid arrangement since end_{i-1} always equals $start_i$. end_0 = 9 == 9 = $start_1$ end_1 = 4 == 4 = $start_2$ end_2 = 5 == 5 = $start_3$*

Example 2

Input	`pairs = [[1,3],[3,2],[2,1]]`
Output	`[[1,3],[3,2],[2,1]]`
Explanation	*This is a valid arrangement since end_{i-1} always equals $start_i$. end_0 = 3 == 3 = $start_1$ end_1 = 2 == 2 = $start_2$ The arrangements [[2,1],[1,3],[3,2]] and [[3,2],[2,1],[1,3]] are also valid.*

Example 3

Input	`pairs = [[1,2],[1,3],[2,1]]`
Output	`[[1,2],[2,1],[1,3]]`
Explanation	*This is a valid arrangement since end_{i-1} always equals $start_i$. end_0 = 2 == 2 = $start_1$ end_1 = 1 == 1 = $start_2$*

CONSTRAINTS

- 1 <= pairs.length <= 10^5
- pairs[i].length == 2
- 0 <= $start_i$, end_i <= 10^9
- $start_i$!= end_i
- No two pairs are exactly the same.
- There exists a valid arrangement of pairs.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Arrange pairs so each pair's end connects to the next pair's start. Return valid arrangement. Right?"

"A few quick questions:

This is an Eulerian path problem: each pair is a directed edge; find a path traversing all edges once."

"Let me walk: pairs=[[5,1],[4,5],[11,9],[9,4]] -> [[11,9],[9,4],[4,5],[5,1]] ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(pairs):
    check that each pair's end equals the next
        pair's start
return the first valid one
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. The pairs are edges of a graph and the arrangement is an Eulerian path: Hierholzer's algorithm (start at the node with out-degree minus in-degree = 1, else any) builds it in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Hierholzer's for Eulerian path on directed graph — O(E log E). Walk-through:"

Directed Eulerian path: each node's in-degree must equal out-degree (Eulerian circuit here).

Build directed adjacency: for each pair [start, end]: graph[start].append(end).

Sort adjacency lists (lexicographic edge order).

Hierholzer's DFS from any start node:

While current node has unused edges: follow next edge (pop from sorted list).

When stuck: append current node to result. Backtrack.

Reverse result -> valid Euler path.

WHY post-order append: a dead-end node (no more outgoing) goes LAST in the path.

Hierholzer guarantees every edge is traversed exactly once.

Time: O(E log E) for sorting. Space: O(E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
class _2097_ValidArrangementOfPairs:
    def validArrangement(self, pairs):
        graph = defaultdict(list)
        in_deg = defaultdict(int)
        out_deg = defaultdict(int)
        for u, v in pairs:
            graph[u].append(v)
            out_deg[u] += 1
            in_deg[v] += 1
        start = pairs[0][0]
        for node in set(out_deg) | set(in_deg):
            if out_deg[node] - in_deg.get(node, 0) == 1:
                start = node
                break
        path = []
        stack = [start]
        while stack:
            while graph[stack[-1]]:
                stack.append(graph[stack[-1]].pop())
            path.append(stack.pop())
        path.reverse()
        return [[path[i], path[i+1]] for i in range(len(pa
th)-1)]
```

PHASE 5 TEST & DEBUG

"Let me trace validArrangement([[5,1],[4,5],[11,10],[10,4],[10,11]])."

Eulerian path on directed graph: each pair [u,v] is directed edge u->v

In-degree vs out-degree: find node with out-degree - in-degree = 1 (start)

Node 5: out=1(->1),in=1(4->5) -> balanced. Node 4: out=1(->5),in=1(10->4).

Node 10: out=2(->4,->11),in=1(11->10). out-in=1 -> start at 10? No: 11->10,10->4,10->11->10->4...

Find start: out[v]-in[v]=1 -> or any node if all balanced (circuit)

Hierholzer's: build Eulerian path = [5,1...] or [10,4,5,1... wait

Result: [[10,11],[11,10],[10,4],[4,5],[5,1]] ok

Eulerian circuit: component must be connected. find(start)->node with out-in-degree=1. Stack-based Hierholzer: component = entire directed graph. cost=O(E log E) for sorting.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(E)	O(V+E)

Follow-up: Cracking the Safe (#753) — de Bruijn sequence uses same Eulerian circuit.

Follow-up: Reconstruct Itinerary (#332) — same Hierholzer's on flight graph.

HARD | DFS | LeetCode #2392 | 11 of 12 | 11/12

Build a Matrix With Conditions

QUESTION

You are given a positive integer k. You are also given:

CONSTRAINTS

- a 2D integer array rowConditions of size n where rowConditions[i] = [above$_i$, below$_i$], and
- a 2D integer array colConditions of size m where colConditions[i] = [left$_i$, right$_i$].

The two arrays contain integers from 1 to k.

You have to build a k x k matrix that contains each of the numbers from 1 to k exactly once. The remaining cells should have the value 0.

The matrix should also satisfy the following conditions:

- The number above$_i$ should appear in a row that is strictly above the row at which the number below$_i$ appears for all i from 0 to n - 1.
- The number left$_i$ should appear in a column that is strictly left of the column at which the number right$_i$ appears for all i from 0 to m - 1.

Return any matrix that satisfies the conditions. If no answer exists, return an empty matrix.

Example 1

Input	`k = 3, rowConditions = [[1,2],[3,2]], colConditions = [[2,1],[3,2]]`
Output	`[[3,0,0],[0,0,1],[0,2,0]]`
Explanation	*The diagram above shows a valid example of a matrix that satisfies all the conditions.* *The row conditions are the following:* *- Number 1 is in row 1, and number 2 is in row 2, so 1 is above 2 in the matrix.* *- Number 3 is in row 0, and number 2 is in row 2, so 3 is above 2 in the matrix.* *The column conditions are the following:* *- Number 2 is in column 1, and number 1 is in column 2, so 2 is left of 1 in the matrix.* *- Number 3 is in column 0, and number 2 is in column 1, so 3 is left of 2 in the matrix.* *Note that there may be multiple correct answers.*

QUESTION

Example 2

Input	`k = 3, rowConditions = [[1,2],[2,3],[3,1],[2,3]], colConditions = [[2,1]]`
Output	`[]`
Explanation	*From the first two conditions, 3 has to be below 1 but the third conditions needs 3 to be above 1 to be satisfied.* *No matrix can satisfy all the conditions, so we return the empty matrix.*

- 2 <= k <= 400
- 1 <= rowConditions.length, colConditions.length <= 10^4
- rowConditions[i].length == colConditions[i].length == 2
- 1 <= above$_i$, below$_i$, left$_i$, right$_i$ <= k
- above$_i$!= below$_i$
- left$_i$!= right$_i$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Place numbers 1..k in a k×k matrix such that rowConditions and colConditions are satisfied:

each [a,b] means a must appear in an earlier row/column than b. Return any valid matrix, or empty if impossible. Right?"

"A few quick questions:

Each number appears exactly once in the matrix? Yes. Conditions form a DAG (Directed Acyclic Graph) if satisfiable (need topological sort)."

"Let me walk: k=3, rowConditions=[[1,2],[3,2]], colConditions=[[2,1]] -> valid matrix exists ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to place the numbers 1 to k in the k x k matrix
    check the row and column conditions
return the first valid placement
```

TIME	SPACE
O(k! * k^2)	O(k^2)

WHY IT IS SLOW

Enumerates placements. Topologically sort the numbers by the row conditions (giving each number's row) and again by the column conditions (its column); a cycle in either means no solution: O(k + conditions).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two independent topological sorts (rows then columns) — O(k + E). Walk-through:"

Step 1 — Row order: build directed graph from rowConditions.

Edge (a,b) means row of a must come before row of b.

Run Kahn's BFS topo sort on values 1..k. Get row_order[v] = row index for value v.

If cycle -> return [].

Step 2 — Column order: same with colConditions. Get col_order[v].

Step 3 — Build k×k matrix:

`matrix[row_order[v]][col_order[v]] = v for each value v in 1..k.`

WHY two independent sorts work: row and column constraints are orthogonal —

satisfying row ordering doesn't conflict with column ordering.

Each value occupies a unique (row, col) position determined by its relative rank in each dimension.

Time: O(k + E). Space: O(k + E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _2392_BuildAMatrixWithConditions:
    def buildMatrix(self, k, rowConditions, colConditions)
:
        def topological_order(conditions):
            graph = defaultdict(list)
            in_degree = [0] * (k + 1)
            for a, b in conditions:
                graph[a].append(b)
                in_degree[b] += 1
            queue = deque(i for i in range(1, k + 1) if in
_degree[i] == 0)
            order = []
            while queue:
                node = queue.popleft()
                order.append(node)
                for neighbor in graph[node]:
                    in_degree[neighbor] -= 1
                    if in_degree[neighbor] == 0:
                        queue.append(neighbor)
            return order if len(order) == k else []

        row_order = topological_order(rowConditions)
        col_order = topological_order(colConditions)
        if not row_order or not col_order:
            return []

        row_index = {num: i for i, num in enumerate(row_or
der)}
        col_index = {num: i for i, num in enumerate(col_or
der)}
        matrix = [[0] * k for _ in range(k)]
        for num in range(1, k + 1):
            matrix[row_index[num]][col_index[num]] = num
        return matrix
```

PHASE 5 TEST & DEBUG

"Let me trace the graph traversal, and show how visited prevents cycles."

k=3, rowConditions=[[1,2],[3,2]], colConditions=[[2,1]]:

row topo: in_degree[2]=2,others 0. queue=[1,3]. pop1->order=[1]. pop3->order=[1,3]. 2's indeg drops to 0 twice->queue=[2].

pop2->order=[1,3,2]. row_index={1:0,3:1,2:2}.

col topo: [[2,1]]: in_degree[1]=1. queue=[2,3]. order builds to [2,3,1] or [3,2,1] (some valid order).

Build matrix placing each number at (row_index[num], col_index[num]) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k + E)	O(k + E)

Follow-up: Course Schedule II (#210) — single topological sort, simpler 1D output.

Follow-up: if conditions conflict (cycle), explain which approach (DFS-based cycle detection) gives a faster fail-fast.

HARD | DFS | LeetCode #2709 | 12 of 12 | 12/12

Greatest Common Divisor Traversal

QUESTION

You are given a 0-indexed integer array nums, and you are allowed to traverse between its indices. You can traverse between index i and index j, i != j, if and only if gcd(nums[i], nums[j]) > 1, where gcd is the greatest common divisor.

Your task is to determine if for every pair of indices i and j in nums, where i < j, there exists a sequence of traversals that can take us from i to j.

Return true if it is possible to traverse between all such pairs of indices, or false otherwise.

Example 1

Input	`nums = [2,3,6]`
Output	`true`
Explanation	*In this example, there are 3 possible pairs of indices: (0, 1), (0, 2), and (1, 2). To go from index 0 to index 1, we can use the sequence of traversals 0 -> 2 -> 1, where we move from index 0 to index 2 because gcd(nums[0], nums[2]) = gcd(2, 6) = 2 > 1, and then move from index 2 to index 1 because gcd(nums[2], nums[1]) = gcd(6, 3) = 3 > 1. To go from index 0 to index 2, we can just go directly because gcd(nums[0], nums[2]) = gcd(2, 6) = 2 > 1. Likewise, to go from index 1 to index 2, we can just go directly because gcd(nums[1], nums[2]) = gcd(3, 6) = 3 > 1.*

Example 2

Input	`nums = [3,9,5]`
Output	`false`
Explanation	*No sequence of traversals can take us from index 0 to index 2 in this example. So, we return false.*

Example 3

Input	`nums = [4,3,12,8]`
Output	`true`
Explanation	*There are 6 possible pairs of indices to traverse between: (0, 1), (0, 2), (0, 3), (1, 2), (1, 3), and (2, 3). A valid sequence of traversals exists for each pair, so we return true.*

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two indices i,j are 'connected' if gcd(nums[i],nums[j]) > 1. Determine if ALL pairs of indices

are reachable from each other via a chain of such connections (i.e. the graph is fully connected). Right?"

"A few quick questions:

Connect indices through SHARED PRIME FACTORS rather than checking every pair directly (too slow)? Yes — that's the key insight."

"Let me walk: nums=[2,3,6] -> 2&6 share factor 2, 3&6 share factor 3 -> all connected via 6 -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build the complete graph on the indices: connect i
    and j when gcd(nums[i], nums[j]) > 1
check that it is connected
```

TIME	SPACE
O(n^2 * log(max))	O(n^2)

WHY IT IS SLOW

Computes a gcd for every pair. Union each index with the prime factors of its value (union-find over primes); the graph is connected when all values end up in one component: about O(n * sqrt(max)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is O(n²) pairwise gcd checks.

Instead, union each index with ITS PRIME FACTORS as virtual nodes. Two indices sharing a prime

factor get unioned transitively through that prime's virtual node. Factorize each number in

O(sqrt(val)) time, union index with each prime factor.

Time: O(n*sqrt(max_val)*alpha(n)). Space: O(n + distinct primes)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

Key insight: routing union-find edges through shared prime factor virtual nodes gives O(nV) vs O(n²) direct GCD pairs — two indices connect iff any prime factor is shared.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2709_GreatestCommonDivisorTraversal:
    def canTraverseAllPairs(self, nums):
        n = len(nums)
        if n == 1:
            return True
        if any(num == 1 for num in nums):
            return False
        parent = {}
        def find(x):
            if x not in parent:
                parent[x] = x
            if parent[x] != x:
                parent[x] = find(parent[x])
            return parent[x]
        def union(a, b):
            ra, rb = find(a), find(b)
            if ra != rb:
                parent[ra] = rb
        def prime_factors(x):
            factors = []
            d = 2
            while d * d <= x:
                if x % d == 0:
                    factors.append(d)
                    while x % d == 0:
                        x //= d
                d += 1
            if x > 1:
                factors.append(x)
            return factors

        for i, num in enumerate(nums):
            for p in prime_factors(num):
                union(('idx', i), ('prime', p))

        roots = {find(('idx', i)) for i in range(n)}
        return len(roots) == 1
```

PHASE 5 TEST & DEBUG

"Let me trace the graph traversal, and show how visited prevents cycles."

nums=[2,3,6]: prime_factors(2)=[2]. union(idx0,prime2). prime_factors(3)=[3]. union(idx1,prime3).

prime_factors(6)=[2,3]. union(idx2,prime2)->idx2 joins idx0's group via prime2.

union(idx2,prime3)-> idx2's group merges with idx1's group via prime3.

All three indices end up in the same root -> True ok

nums=[3,9,5]: prime_factors give {3} for idx0,idx1; {5} for idx2. idx2 never connects -> False ok

Using primes as virtual union-find nodes lets us connect indices in O(sqrt(val)) per number instead of O(n) pairwise comparisons

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*sqrt(max_val)*alpha(n))	O(n + distinct primes)

Follow-up: Number of Provinces (#547) — simpler union-find without the prime-factor trick.

Follow-up: Smallest String With Swaps (#1202) — same union-find-via-shared-property idea.

IN YOUR APP | Expose | Editorial | Community